Design & Analysis of Algorithms

Unit 2 · Exam notes in easy language · M23DE0303 (MCA Sem III, REVA University) · From Dr. Lakshmi K's slides (Levitin textbook). Results marked "standard" are textbook facts added to complete the topic.

> **This deck in 4 lines**
> **Brute force** = try the obvious way straight from the definition (selection sort, bubble sort, simple string matching, trying every possibility). **Divide and conquer** = split the problem, solve the parts, combine the answers (merge sort, quick sort). Loops are analysed by summation, divide-and-conquer by a recurrence and the **Master Theorem**. Know the algorithm, one traced example, and the complexity of each.

1. Brute force

Brute force is a straightforward approach, usually based directly on the problem statement and the definitions of the concepts involved. You do exactly what the definition says, with no clever tricks.

Examples from the slides (all taken straight from definitions):

- Computing a^n ($a > 0$, n a non-negative integer): multiply a by itself n times.
- Computing n!: multiply $1 \times 2 \times \ldots \times n$.
- Multiplying two matrices: use the row-times-column definition.
- Searching for a key in a list: check items one by one.

Simple to write and always correct, but often slow for large inputs. The slides' brute-force algorithms are below.

2. Selection sort (brute-force sorting)

Idea: repeatedly pick the **smallest** remaining element and put it at the front.

- Scan the whole array, find the smallest, swap it with the first element.
- Then scan from the second element onward, find the smallest of those, swap it with the second element.
- In general, on pass i ($0 \le i \le n-2$): find the smallest in A[i..n−1] and swap it with A[i]. Everything left of position i is already in its final place.

```
SelectionSort(A[0..n−1])
for i ← 0 to n−2 do
    min ← i
    for j ← i+1 to n−1 do
        if A[j] < A[min]  then  min ← j
    swap A[i] and A[min]
```

Trace on 89 45 68 90 29 34 17 (after each pass; the sorted part grows from the left):

Pass	Smallest found	Array after the swap
start		89 45 68 90 29 34 17
1	17	**17** | 45 68 90 29 34 89

Pass	Smallest found	Array after the swap
2	29	**17 29** | 68 90 45 34 89
3	34	**17 29 34** | 90 45 68 89
4	45	**17 29 34 45** | 90 68 89
5	68	**17 29 34 45 68** | 90 89
6	89	**17 29 34 45 68 89 90**

Analysis. Basic operation: the comparison A[j] < A[min]. It does not depend on how the data is arranged.

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = (n-1)n / 2 \in \Theta(n^2)$$

Number of key **swaps** = n − 1, which is **Θ(n)**. So selection sort is quadratic in comparisons but very light on swaps.

3. Bubble sort

Idea: compare **neighbouring** elements and swap them if they are out of order. Each pass "bubbles up" the largest remaining element to the end. After pass i, the last i elements are in their final positions.

```
BubbleSort(A[0..n−1])
for i ← 0 to n−2 do
    for j ← 0 to n−2−i do
        if A[j+1] < A[j]  then  swap A[j] and A[j+1]
```

Trace on 89 45 68 90 29 34 17 (array at the end of each pass; the part after the bar is final):

Pass	Array at end of pass
1	45 68 89 29 34 17 | **90**
2	45 68 29 34 17 | **89 90**
3	45 29 34 17 | **68 89 90**
4	29 34 17 | **45 68 89 90**
5	29 17 | **34 45 68 89 90**
6	**17 29 34 45 68 89 90**

Analysis. Basic operation: the comparison A[j+1] < A[j]. Pass i makes (n−1−i) comparisons, so

$$C(n) = \sum_{i=0}^{n-2} (n-1-i) = n(n-1)/2 \in \Theta(n^2)$$

Number of swaps in the **worst case** (list in reverse order) = number of comparisons: $S_{worst}(n) = C(n) = n(n-1)/2$.

4. Sequential search and brute-force string matching

Sequential search

```
SequentialSearch(A[0..n−1], K)
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
if i < n  return i   else  return −1
```

Input size: n. Basic operation: the comparison (i < n, A[i] ≠ K). Worst case: $\mathbf{C_{worst}(n) = n}$ (key at the end, or absent). It returns the index of the first match, or −1.

Brute-force string matching

Pattern = a string of m characters to find. **Text** = a longer string of n characters to search in. **Goal:** find a substring of the text that matches the pattern.

1. **Step 1:** align the pattern at the beginning of the text.
2. **Step 2:** moving left to right, compare each pattern character with the matching text character until either **all m match** (success) or a **mismatch** is found.
3. **Step 3:** while the pattern is not found and the text is not finished, slide the pattern **one position right** and repeat step 2.

```
BruteForceStringMatch(T[0..n−1], P[0..m−1])
for i ← 0 to n−m do
    j ← 0
    while j < m and P[j] = T[i+j] do
        j ← j + 1
    if j = m  return i      // index where the match starts
return −1
```

Slide example: text = N O B O D Y _ N O T I C E D _ H I M, pattern = N O T. The pattern is tried at every position: at position 0, N and O match but T ≠ B stops it; it slides right one step at a time, until at position 7 all of N O T match.

> **Time efficiency: Θ(mn) comparisons in the worst case**
> There are about (n − m + 1) alignments and each may compare up to m characters. A bad case (standard): text = aaaa...a, pattern = aaab. Every alignment compares m characters before failing. For ordinary text, mismatches come early, so it is usually much faster.

5. Exhaustive search

Exhaustive search is the brute-force approach for **combinatorial and optimization** problems: **generate every possible candidate, check each one, keep the best**. It is correct but the number of candidates explodes.

(a) Travelling Salesman Problem (TSP)

Visit every city exactly once and return to the start with the **shortest total distance**. It is the same as finding the **shortest Hamiltonian circuit** (a cycle that visits every vertex once).

Slide example: four cities a, b, c, d with distances ab = 2, ac = 5, ad = 7, bc = 8, bd = 3, cd = 1. List every tour:

Tour	Length
a → b → c → d → a	2 + 8 + 1 + 7 = 18
a → b → d → c → a	2 + 3 + 1 + 5 = **11 (optimal)**
a → c → b → d → a	5 + 8 + 3 + 7 = 23
a → c → d → b → a	5 + 1 + 3 + 2 = **11 (optimal)**
a → d → b → c → a	7 + 3 + 8 + 5 = 23
a → d → c → b → a	7 + 1 + 8 + 2 = 18

The best tour has length 11. (The two optimal tours are the same cycle read in opposite directions.) With the start fixed, there are (n − 1)! tours (standard), so the work grows like a factorial.

(b) Knapsack problem

Given n items with weights w_1, ..., w_n and values v_1, ..., v_n, and a knapsack of capacity W, find the **most valuable subset of items that fits** (total weight ≤ W).

Slide example (W = 10): item 1: weight 7, value $42 · item 2: weight 3, value $12 · item 3: weight 4, value $40 · item 4: weight 5, value $25. Try every subset:

Subset	Weight	Value	Subset	Weight	Value
ø	0	$0	{2, 3}	7	$52
{1}	7	$42	{2, 4}	8	$37
{2}	3	$12	{3, 4}	9	**$65 ← best**
{3}	4	$40	{1, 2}	10	$54
{4}	5	$25	{1, 3}, {1, 4}, {1, 2, 3}, and every larger subset	over 10	not feasible

Best choice: items {3, 4}, weight 9, value **$65**. With n items there are 2^n subsets (standard), so the work is exponential.

(c) Assignment problem

There are n people and n jobs. Person i doing job j costs C[i, j]. Give **each person exactly one job** so that the **total cost is minimum**.

Slide example (cost matrix; rows = persons, columns = jobs):

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

Each possible assignment is a **permutation** of the jobs, written ⟨job of person 1, job of person 2, ...⟩. Examples from the slide: ⟨1,2,3,4⟩ costs 9+4+1+4 = 18; ⟨1,2,4,3⟩ costs 9+4+8+9 = 30; ⟨1,3,2,4⟩ costs 9+3+8+4 = 24; and so on for all of them.

Time complexity: n!, because there are n! permutations. The slide lists only the first few. Checking all 24, the cheapest is ⟨2,1,3,4⟩ = 2 + 6 + 1 + 4 = **13**.

6. Divide and conquer

The best-known design strategy:

1. **Divide** the problem into two or more smaller instances.
2. **Conquer:** solve the smaller instances (usually by recursion).
3. **Combine** their solutions into the solution of the original problem.

Typical picture: a problem of size n becomes two sub-problems of size n/2, each is solved, and the two solutions are combined.

Divide-and-conquer vs decrease-and-conquer: the difference is somewhat vague. Decrease-and-conquer typically does **not** solve both sub-problems, it solves only one smaller problem.

Examples: merge sort and quick sort; binary tree traversals; multiplication of large integers; Strassen's matrix multiplication; closest-pair and convex-hull algorithms. **Binary search** is "decrease by half", a degenerate form of divide and conquer.

General recurrence and the Master Theorem

$$T(n) = a \cdot T(n/b) + f(n), \quad \text{where } f(n) \in \Theta(n^d),\ d \geq 0$$

a = number of sub-problems, b = factor by which size shrinks, f(n) = cost of dividing and combining.

Compare a with b^d	Result	Same thing using logs
$a < b^d$	$\mathbf{T(n) \in \Theta(n^d)}$	$\log_b a < d$
$a = b^d$	$\mathbf{T(n) \in \Theta(n^d \log n)}$	$\log_b a = d$
$a > b^d$	$\mathbf{T(n) \in \Theta(n^{\log_b a})}$	$\log_b a > d$

The same results hold with O instead of Θ. Memory trick: **work at the top wins → n^d; equal → $n^d \log n$; work at the bottom (many small problems) wins → $n^{\log_b a}$**.

The three examples on the slide (all have a = 4, b = 2):

Recurrence	d	b^d	Compare	Answer
$T(n) = 4T(n/2) + n$	1	2	$a = 4 > 2$	$\mathbf{\Theta(n^2)}$ ($\log_2 4 = 2$)
$T(n) = 4T(n/2) + n^2$	2	4	$a = 4 = 4$	$\mathbf{\Theta(n^2 \log n)}$
$T(n) = 4T(n/2) + n^3$	3	8	$a = 4 < 8$	$\mathbf{\Theta(n^3)}$

7. Merge sort

Idea: split the array in half, sort each half recursively, then **merge** the two sorted halves into one sorted array.

1. Split A[0..n−1] into two about equal halves and copy them into arrays B and C.
2. Sort B and C recursively.
3. **Merge**: repeat until one array is used up: compare the first unused elements of B and C, copy the smaller into A and move that array's pointer forward. Then copy the leftovers of the other array into A.

```
Mergesort(A[0..n−1])
if n > 1
    copy A[0..⌊n/2⌋−1] to B[0..⌊n/2⌋−1]
    copy A[⌊n/2⌋..n−1] to C[0..⌈n/2⌉−1]
    Mergesort(B);  Mergesort(C)
    Merge(B, C, A)
```

```
Merge(B[0..p−1], C[0..q−1], A[0..p+q−1])
i ← 0;  j ← 0;  k ← 0
while i < p and j < q do
    if B[i] ≤ C[j]   then  A[k] ← B[i];  i ← i + 1
    else                   A[k] ← C[j];  j ← j + 1
    k ← k + 1
if i = p  copy C[j..q−1] to A[k..p+q−1]
else      copy B[i..p−1] to A[k..p+q−1]
```

Thinking about recursive programs (slide): assume the recursive calls work, then ask whether the whole program works.

Worked example (slide): 8 3 2 9 7 1 5 4

Step	What happens
Split	8 3 2 9 7 1 5 4 → [8 3 2 9] and [7 1 5 4] → [8 3] [2 9] [7 1] [5 4] → single elements
Merge level 1	[3 8] [2 9] [1 7] [4 5]
Merge level 2	[2 3 8 9] [1 4 5 7]
Merge level 3	**1 2 3 4 5 7 8 9**

Merging example (from the teacher's notes): B = (1 2 5 7 9), C = (3 4 6). Compare 1 & 3 → 1; 2 & 3 → 2; 5 & 3 → 3; 5 & 4 → 4; 5 & 6 → 5; 7 & 6 → 6. C is used up, so copy the rest of B: 7 9. Result: **1 2 3 4 5 6 7 9**.

Analysis

Basic operation: key comparison. Merging two halves needs at most n − 1 comparisons in the worst case ($C'(n) = n - 1$):

$$C(n) = 2C(n/2) + (n - 1), \quad C(1) = 0$$

Master Theorem: a = 2, b = 2, f(n) = n so d = 1. Then $a = b^d$ (2 = 2) → **$C(n) \in \Theta(n \log n)$**. For $n = 2^k$ the exact answer is $n \log_2 n - n + 1$ (standard).

- **All cases** (best, average, worst) are Θ(n log n).
- The worst-case comparisons are close to the theoretical minimum for any comparison-based sort: $\log_2 n! \approx n \log_2 n - 1.44n$.
- **Space:** the book's version uses about n + n(1 + ½ + ¼ + ...) = Θ(3n) extra space. It can be cut to Θ(2n) by not copying during sorting (pass index bounds, copy into one extra array only while merging, then copy back). It can even be done in place, and without recursion (bottom-up).

8. Quick sort

Quick sort (also called **partition-exchange sort**) puts one element, the **pivot**, into its correct final position, and splits the array into two parts around it. Then each part is sorted the same way. "Proper position" means the position the element would have in the fully sorted array.

It is the combination of two algorithms: **Quicksort** (the recursion) and **Partition** (the rearranging).

Partition: using the first element as pivot

Rearrange so that **everything left of the pivot is ≤ pivot** and **everything right of it is > pivot**:

A[0] ... A[s−1] | **A[s]** (pivot) | A[s+1] ... A[n−1] (left part ≤ A[s], right part ≥ A[s])

Two pointers scan: **i** moves right from the left end, **j** moves left from the right end.

- i stops at an element **bigger** than the pivot; j stops at an element **not bigger** than the pivot.
- If i < j (not crossed yet): swap A[i] and A[j], and keep scanning.
- If i > j (crossed): swap the pivot with A[j]. The pivot is now in its final place at position s = j.
- If i = j: they point to an element equal to the pivot, so s = i = j.

```
Quicksort(A[low..high])
if low < high
    s ← Partition(A[low..high])    // s = final position of the pivot
    Quicksort(A[low..s−1])
    Quicksort(A[s+1..high])
```

```
Partition(A[low..high])
pivot ← A[low];   i ← low + 1;   j ← high
loop
    while i ≤ high and A[i] ≤ pivot:  i ← i + 1     // stop at an element > pivot
    while A[j] > pivot:               j ← j − 1     // stop at an element ≤ pivot
    if i ≥ j  then break
    swap A[i] and A[j]
swap A[low] and A[j]
return j
```

> **Slide mistake**
> The partition code on slide 40 says "repeat i ← i+1 **until A[i] ≤ pivot**" and "repeat j ← j−1 **until A[j] > pivot**". Those two conditions are reversed. They contradict the slide's own rule (left ≤ pivot, right > pivot). The version above is the correct one: i skips elements ≤ pivot, j skips elements > pivot.

Worked example: 5 3 1 9 8 2 4 7 (the slide's example)

Pivot = 5. i stops at 9 (position 3), j stops at 4 (position 6) → swap: 5 3 1 **4** 8 2 **9** 7. Next i stops at 8, j stops at 2 → swap: 5 3 1 4 **2 8** 9 7. Now i (at 8) has crossed j (at 2). Swap the pivot with A[j]:

2 3 1 4 | **5** | 8 9 7 (s = 4)

Then quick sort the left part (2 3 1 4) and the right part (8 9 7) in the same way. The slide's tree of calls starts with l = 0, r = 7, s = 4, then splits into l = 0..3 and l = 5..7.

Exercises from the slide, solved

- **24, 14, 12, 36, 8, 55, 29, 6:** first partition (pivot 24) gives 8 14 12 6 | **24** | 55 29 36, so s = 4. Final sorted list: **6, 8, 12, 14, 24, 29, 36, 55**.
- **QUICK SOSRT** (letters Q U I C K S O S R T): first partition (pivot Q) gives K O I C | **Q** | S U S R T. Final sorted list: **C, I, K, O, Q, R, S, S, T, U**.

Time complexity of quick sort

Case	When	Count	Class
Best	Pivot always lands in the middle	$C_{best}(n) = 2C_{best}(n/2) + n$, $C_{best}(1) = 0$. Master Theorem ($a = 2, b = 2, d = 1$): equal case. For $n = 2^k$, $C_{best}(n) = n \log_2 n$	**$\Theta(n \log n)$**
Worst	Pivot is always the smallest or largest, e.g. an already sorted array (standard)	$C_{worst}(n) = (n+1) + n + \ldots + 3 = (n+1)(n+2)/2 - 3$	**$\Theta(n^2)$**
Average	Pivot lands in any position with equal chance	$C_{avg}(n) = (1/n) \sum_{s=0}^{n-1} [(n+1) + C_{avg}(s) + C_{avg}(n-1-s)]$, $C_{avg}(0) = C_{avg}(1) = 0$. Solving gives about $1.39\, n \log_2 n$ (standard)	**$\Theta(n \log n)$**

Merge sort vs quick sort

The slides list this as an agenda item but do not include the content. This is the standard comparison.

	Merge sort	Quick sort
How it splits	Simply in the middle; the hard work is the **merge**	By value around a pivot; the hard work is the **partition**
Worst case	$\Theta(n \log n)$	$\Theta(n^2)$
Best / average	$\Theta(n \log n)$	$\Theta(n \log n)$
Extra memory	Needs extra arrays (about 2n to 3n)	Works in place (apart from recursion)
Stable?	Yes (when merging with ≤)	No

9. Last-minute revision

Results table (memorise)

Algorithm	Basic operation	Count	Class
Selection sort	A[j] < A[min]	$n(n-1)/2$; swaps = $n-1$	$\Theta(n^2)$; swaps $\Theta(n)$

Algorithm	Basic operation	Count	Class
Bubble sort	A[j+1] < A[j]	n(n−1)/2; worst swaps = same	$\Theta(n^2)$
Sequential search	Key comparison	worst n	O(n)
Brute-force string match	Character comparison	up to m per alignment, n−m+1 alignments	Θ(mn) worst
TSP / knapsack / assignment	Check one candidate	tours / 2^n subsets / n! permutations	factorial or exponential
Merge sort	Key comparison	C(n) = 2C(n/2) + n − 1	Θ(n log n), all cases
Quick sort	Key comparison	best 2C(n/2)+n; worst (n+1) (n+2)/2 − 3	best/avg Θ(n log n), worst $\Theta(n^2)$

Master Theorem in one line

$$T(n) = aT(n/b) + \Theta(n^d): \quad a < b^d \to \Theta(n^d) \cdot a = b^d \to \Theta(n^d \log n) \cdot a > b^d \to \Theta(n^{\log_b a})$$

Questions likely to come

1. What is brute force? Give examples.
2. Write the selection sort algorithm, trace it on a list, and derive C(n) = n(n−1)/2 and the number of swaps.
3. Write the bubble sort algorithm, trace it, and analyse it.
4. Explain brute-force string matching with an example. What is its worst-case time?
5. What is exhaustive search? Solve a small TSP, knapsack and assignment instance.
6. Explain the divide-and-conquer technique. How does it differ from decrease-and-conquer?
7. State the Master Theorem and solve T(n) = 4T(n/2) + n, + n^2, + n^3.
8. Write the merge sort and merge algorithms; sort a list step by step; derive its recurrence and Θ(n log n).
9. Discuss the space needed by merge sort.
10. Write quick sort and partition; sort a list (e.g. 24 14 12 36 8 55 29 6).
11. Analyse quick sort for best, worst and average cases.
12. Differences between merge sort and quick sort.

Traps and slide mistakes

- The quick sort partition pseudocode on the slides has its two scanning conditions reversed. Use the corrected version in section 8.
- Selection sort does only n − 1 swaps (Θ(n)), but still $\Theta(n^2)$ comparisons. Bubble sort's worst-case swaps equal its comparisons.
- Merge sort is Θ(n log n) in **every** case; quick sort is $\Theta(n^2)$ in the worst case.
- In the Master Theorem, compare **a with b^d**, not with d.
- The unit outline (slide 2) lists **insertion sort, DFS, BFS, topological sorting, Horspool's and Boyer-Moore string matching** and binary search, but this deck has no slides on them (binary search appears only as a one-line example). They are not in these notes.